Learn VGGT — Architecture, Attention, Math & Training

Static teaching content extracted from the *Learn VGGT* panel of the VGGT-proto app. Excludes interactive widgets that need a running model (attention picker on your scene, geometry sliders driving the 3D viewer). Sources: *"VGGT: Visual Geometry Grounded Transformer", Wang et al., CVPR 2025*; public `github.com/facebookresearch/vggt` ; this app's `web/learn.js` .

1. Architecture overview

VGGT is a feed-forward transformer that takes 1–N RGB images and predicts cameras, dense depth + point maps, and 2D point tracks **in a single forward pass**. The pipeline:

1. Patch-tokenize each image with a DINOv2 ViT-L/14 backbone; prepend a camera token and 4 register tokens per image.
2. Run the **Alternating-Attention Aggregator** — about 24 blocks alternating *frame-wise* ↔ *global* self-attention.
3. Pass aggregator tokens to four heads: camera, DPT depth, DPT point, track.
4. Output extrinsics+intrinsics, dense depth + confidence, dense point map + confidence, and 2D tracks + visibilities — all in the **camera-1 world frame**.

1.1 Input & coordinate frame

VGGT ingests a batched stack of N views as one tensor $\mathbf{I} \in \mathbb{R}^{B\times N\times 3\times H\times W}$ with $B = 1$ and $H, W \equiv 0 \pmod{14}$. Preprocessing (in `vggt/utils/load_fn.py`) resizes, pads to a multiple of the patch size 14, and normalizes to DINOv2 mean/std. EXIF orientation is applied by the host app (see `backend/app.py`) — the model itself does not honor EXIF tags.

Coordinate frame. All extrinsics, depth, and point predictions are expressed in the world frame defined as **camera 1 = identity**. The first camera token is special-cased inside the aggregator (it anchors the global frame). That is also why the tracker's "fast path" only works when the reference frame is index 0 — the retained tokens already live in that frame.

1.2 Tokenization (DINOv2 ViT-L/14 + special tokens)

> **Common misconception — DINOv2 is initialization, not a separate stage.** VGGT does *not* "run DINOv2 first, then run its own model". DINOv2's pretrained weights are **copied into** VGGT's patch-embedding conv (and the early aggregator transformer layers). VGGT then keeps training those parameters with its own multi-task loss. At inference there is one network — its patch-embed weights happen to *originate* from DINOv2.

"Pretrained" ≠ "frozen" ≠ "fixed" — three distinct ideas

term	meaning	applies to VGGT's patch embed?
pretrained	initialized from weights learned on a previous task (here: DINOv2's self-supervised objective)	✓ yes
frozen	weights don't update during the current task's training (no gradient flow)	× no — VGGT fine-tunes end-to-end
fixed	doesn't change at all, ever	× no (except trivially at inference, like every weight)

So the patch-embedding conv is **a learnable transformation, pretrained on DINOv2, then fine-tuned end-to-end with VGGT's multi-task loss**. At inference of course all weights are static — but that's true of every parameter, not something special about the patch embed.

Aside: some recipes *do* freeze the backbone — called "linear probing" or "partial freezing". VGGT's paper doesn't suggest that; standard interpretation is full end-to-end fine-tuning.

1. Patch embedding. A single conv strides over the image:

$$\text{PatchEmbed}: \ \text{Conv2d}(3 \to C,\ k = 14,\ s = 14), \quad C \approx 1024$$

so an image $(3, H, W)$ becomes $P = (H/14)(W/14)$ **patch tokens** of dim C. Weights are initialized from **DINOv2 ViT-L/14** (self-supervised, "registers" variant). The image alone therefore produces P tokens — **not the full** S; the next step prepends 5 more.

2. Special tokens (per image). The aggregator prepends:

- 1× **camera token** — later read by the camera head.
- 4× **register tokens** — absorb high-norm artefacts (Darcet et al. 2023); they exist so patch tokens stay clean.

Total length per image: $S = 1_{\text{cam}} + 4_{\text{reg}} + P$. The aggregator returns the patch start index $\texttt{ps\_idx} = 5$ so heads know where patch tokens begin. The batched layout used internally is

$$\mathbf{X} \in \mathbb{R}^{B \times N \times S \times C}.$$

3. Positional info. 2-D sinusoidal / DINOv2 positional embeddings are added to patch tokens; the camera and register tokens get their own learned positional embeddings.

1.3 Alternating-Attention Aggregator (the engine)

Common misconception — each AA block has TWO sub-blocks, not one. A standard transformer block has one (MSA + MLP). A **VGGT AA block** has **two such sub-blocks back-to-back**: a frame-wise (MSA + MLP) and a global (MSA + MLP). So $L \approx 24$ AA blocks contain **48 MSAs and 48 MLPs total**. "Alternating" refers to this within-block alternation (frame ↔ global), not to alternation across the L blocks.

The pre-LN transformer block recipe:

$$\mathbf{x} \leftarrow \mathbf{x} + \mathrm{MSA}(\mathrm{LN}(\mathbf{x}))$$
$$\mathbf{x} \leftarrow \mathbf{x} + \mathrm{MLP}(\mathrm{LN}(\mathbf{x}))$$

with $\mathrm{MSA}(x) = \bigoplus_{h=1}^{H} \mathrm{Attn}(xW_h^Q, xW_h^K, xW_h^V)\, W^O$ and $\mathrm{MLP}(x) = W_2\, \mathrm{GELU}(W_1 x)$ (MLP ratio 4).

The VGGT twist. Each aggregator block contains *two* such sub-blocks back-to-back; the only difference is which tokens are allowed to attend to which. Given input $\mathbf{X} \in \mathbb{R}^{B \times N \times S \times C}$:

```
def aa_block(X):                          // shape (B,N,S,C)
    # --- frame-wise sub-block ---
    Xf = X.reshape(B*N, S, C)              // each image isolated
    Xf = Xf + MSA_frame(LN(Xf))            // mask = block-diagonal
    Xf = Xf + MLP_frame(LN(Xf))
    X  = Xf.reshape(B, N, S, C)
    # --- global sub-block ---
    Xg = X.reshape(B, N*S, C)              // all tokens, all images
    Xg = Xg + MSA_global(LN(Xg))           // full attention
    Xg = Xg + MLP_global(LN(Xg))
    return Xg.reshape(B, N, S, C)
```

Costs. Frame-wise $\mathcal{O}(N \cdot S^2 \cdot C)$; global $\mathcal{O}((NS)^2 \cdot C)$. Alternating avoids paying the global cost twice as often as needed while still fusing views every block.

Stack. The public 1B checkpoint uses $L \approx 24$ AA blocks at ViT-L/14 dimensions $C = 1024$, $h = 16$ heads, MLP ratio 4 — all initialized from DINOv2.

1.4 Camera head — iterative pose regression

Reads only the N camera tokens (one per image) at `ps_idx − 5`: $\mathbf{c} \in \mathbb{R}^{B \times N \times C}$.

1. Cross-view trunk. A small transformer attends across the N camera tokens, so cameras "talk to each other" and the predicted poses become mutually consistent (no per-image picking — the trunk enforces a joint solution).

2. Iterative refinement. The trunk runs $T = 4$ times, each pass predicting a residual update $\Delta\boldsymbol{\theta}_t$ on top of the previous estimate:

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} + \Delta\boldsymbol{\theta}_t, \quad \boldsymbol{\theta} \in \mathbb{R}^9 = [\,\mathbf{t}\,|\,\mathbf{q}\,|\,\mathrm{FOV}_x, \mathrm{FOV}_y\,].$$

Camera 1 is gauge-fixed to identity (its prediction is overwritten).

3. Decoding to $(R, \mathbf{t}, K)$. The quaternion is normalized then expanded; intrinsics use principal point $(W/2, H/2)$ and $f_x = (W/2)/\tan(\mathrm{FOV}_x/2)$. Source: `vggt/utils/pose_enc.py::pose_encoding_to_extri_intri`.

1.5 DPT depth head — token reassembly

Take patch tokens from several aggregator layers; project, reshape $(P, C) \to (C', H/14, W/14)$, and use up/down convs to build a multi-scale feature pyramid at strides $s_\ell \in \{4, 8, 16, 32\}$. A RefineNet fusion combines them coarse→fine into a (C_o, H, W) map, projected to two channels:

$$\hat{d}(u, v) \in \mathbb{R}_{>0}, \qquad \hat{\sigma}(u, v) \in \mathbb{R}_{>0}.$$

Aleatoric confidence. $\hat{\sigma}$ is the Laplacian scale used at training: pixels VGGT is uncertain about (textureless regions, occlusions) are down-weighted by $\mathcal{L} = |\hat{d} - d| \cdot e^{-\hat{\sigma}} + \alpha\hat{\sigma}$. At inference the host app uses $\hat{\sigma}$ as a confidence floor when building the displayed point cloud.

1.6 DPT point head

Identical DPT decoder structure but with a 3-channel output $\hat{\mathbf{X}}(u, v) \in \mathbb{R}^3$ expressed directly in the camera-1 world frame, plus a confidence channel. The paper finds depth → unprojection more accurate for clean point clouds, so this app uses:

$$\mathbf{X}_w = R^\top\big(\hat{d}\, K^{-1}[u, v, 1]^\top - \mathbf{t}\big)$$

(via `vggt/utils/geometry.py::unproject_depth_map_to_point_map`). The point head is still supervised — it stabilizes training.

1.7 Track head — CoTracker-style temporal refinement

Inputs: dense features built from aggregator tokens, the original images, **`ps_idx`**, and a query tensor $\mathbf{Q} \in \mathbb{R}^{B \times Q \times 2}$ of query pixels in frame 0. The algorithm:

1. **Feature pyramids** — per-frame dense feature maps $\mathbf{F}_n$ via DPT-like projections of aggregator tokens.
2. **Cost volumes** — for each query $\mathbf{q}$ encode its feature $\phi(\mathbf{q})$, then per frame compute a local correlation around the current track estimate $\hat{\mathbf{p}}_n$: $\mathbf{c}_n(\Delta) = \langle \phi(\mathbf{q}), \mathbf{F}_n[\hat{\mathbf{p}}_n + \Delta] \rangle$.
3. **Iterative state updates** ($T = 4$ iters): a small transformer treats each query as a temporal state across frames and produces residuals on track and visibility logits.

Outputs: final tracks $\hat{\mathbf{p}} \in \mathbb{R}^{N \times Q \times 2}$ and visibilities $\hat{v} \in [0, 1]^{N \times Q}$.

1.8 Outputs in a single forward pass

Per view, all in the camera-1 world frame:

- $R \in SO(3)$, $\mathbf{t} \in \mathbb{R}^3$ extrinsics; $K \in \mathbb{R}^{3 \times 3}$ intrinsics.
- Dense $\hat{d}(u, v)$ + $\hat{\sigma}_d$.
- Dense $\hat{\mathbf{X}}(u, v)$ + $\hat{\sigma}_p$.
- Tracks $\hat{\mathbf{p}}$ + visibilities $\hat{v}$ (when queries given).

No iterative reconstruction, no bundle adjustment — one feed-forward pass produces a consistent scene.

1.9 Parameter budget — where ~1B trainable params come from

Approximate breakdown for the public VGGT-1B checkpoint. **All parameters are trainable** end-to-end during VGGT training — the "Init source" column tells you only *where the initial values came from*. Exact numbers depend on checkpoint configuration; verify with the snippet below.

Component	~Params	Init source	From DINOv2?
Patch embedding conv (Conv2d 3→C, k=s=14)	~0.6 M	DINOv2 ViT-L/14	✓
Per-patch positional embeddings	~1.4 M	DINOv2	✓
Camera token (1 learnable vector per image)	~1 K	random — VGGT-specific	—
Register tokens (4 learnable vectors per image)	~4 K	DINOv2 with-registers variant	✓
Aggregator — **frame-wise** sub-blocks (24 × ViT-L block: LN, MSA($4C^2$), MLP($8C^2$))	~302 M	DINOv2 ViT-L/14 transformer blocks	✓
Aggregator — **global** sub-blocks (24 × same structure, new for multi-view)	~302 M	random — VGGT-specific	—
Camera head (cross-view trunk + 9-D regressor, T=4 unrolled passes)	~10–30 M	random — VGGT-specific	—
DPT depth head (reassemble + RefineNet fusion)	~80–120 M	random — VGGT-specific	—
DPT point head (same DPT structure)	~80–120 M	random — VGGT-specific	—
Track head (CoTracker-style iterative)	~50–100 M	random — VGGT-specific	—
Final LayerNorms, biases, misc.	~0.1 M	various	—
Total (VGGT-1B)	**≈ 1.0 B**	**≈ 304 M DINOv2 + ≈ 700 M random**	

Roughly **30% of VGGT's params come from DINOv2** — the patch embedding, positional embeddings, register tokens, and the 24 frame-wise transformer sub-blocks of the aggregator. The remaining **~70% are random-initialized VGGT-original**: the global sub-blocks (new for multi-view fusion) and the four heads. All of it then trains end-to-end.

Verify the real numbers on the pod

```
from backend.vggt_runner import get_model
m = get_model()

total     = sum(p.numel() for p in m.parameters())
trainable = sum(p.numel() for p in m.parameters() if p.requires_grad)
print(f"Total: {total/1e6:.1f} M   Trainable: {trainable/1e6:.1f} M")

# Per-top-level-module breakdown
```

```
for name, mod in m.named_children():
    n = sum(p.numel() for p in mod.parameters())
    print(f"  {name:<14s} {n/1e6:7.1f} M")
```

1.10 Inductive biases — what VGGT assumes before seeing any data

An **inductive bias** is the set of built-in architectural assumptions that bias which functions a model tends to learn — before seeing any training data. The "shape of the prior" over hypotheses.

The spectrum

architecture	strength	built-in assumptions
MLP	~none	universal approximator; sees inputs as flat vector
CNN	strong	locality (nearby pixels matter), translation equivariance, parameter sharing
RNN	strong	sequential / temporal — state carries past in strict order
Transformer (attention)	weak	permutation-equivariant by default (until position embeddings); no locality; no translation equivariance

Trade-off. Strong inductive bias → great for *small data* (prior fills the gaps), low ceiling. Weak inductive bias → bad for small data, high ceiling. Famous result: ViT needs ~JFT-300M data to match CNNs on ImageNet; below that, ResNets win. With enough data ViTs surpass them, and even *recover* CNN-like locality patterns in their first heads (Cordonnier et al. 2020) — the prior *emerges* rather than being hard-coded.

Inductive biases that ARE in VGGT

Even though attention itself is bias-light, VGGT's authors bolt on several **structural priors** — each one an explicit "we believe the world works this way":

bias	what it asserts	where in VGGT
Local pixel structure	Nearby pixels belong together; image has 2-D layout.	14×14 patch-embed conv; 2-D positional embeddings.
Transferred semantic features	Natural images have shared low-level / mid-level structure (textures, objects, surfaces).	DINOv2 init of patch embed and early aggregator layers — a *learned* bias bolted on by transfer.
Two-phase view reasoning	Per-image features first, then fuse across views — better than always-global attention.	Alternating frame ↔ global structure of every AA block.
Camera-1 as world frame	A canonical reference frame exists; pose is relative to it.	First camera token special-cased; gauge fix on camera 1.

Register slots	Some token capacity should be reserved as "scratch space" for artefacts.	4 register tokens per image (DINOv2 with-registers variant).
Dense prediction from token grids	Per-pixel outputs share multi-scale spatial structure (FPN-like fusion helps).	DPT reassembly + RefineNet in depth and point heads.
Iterative pose refinement	Camera pose is best estimated by 4 refinement steps, not one shot.	T=4 unrolled passes inside the camera head.
Heteroscedastic noise	Pixel-wise prediction uncertainty varies across the image (textureless vs textured, occlusions).	Aleatoric Laplace loss heads outputting per-pixel σ.
Pinhole imaging	Cameras are pinholes; principal point at image centre; FOV → focal.	9-D pose encoding; `pose_encoding_to_extri_intri` decoder.

Why this matters intuitively. Without these priors, a generic transformer could in principle learn 3D from images — but you'd need orders of magnitude more data, slower convergence, and the model would have to discover "alternating attention helps", "DPT-style multi-scale fusion helps", "iterative pose refinement converges better" etc. from scratch. These biases are how the authors inject 30+ years of multi-view geometry research into the architecture, so the network doesn't re-derive it from pixels.

2. Inside one Alternating-Attention block

Each AA block takes a token grid $\mathbf{X} \in \mathbb{R}^{B\times N\times S\times C}$ and runs two pre-LN transformer sub-blocks back-to-back — frame-wise first, then global. The only difference between them is the reshape applied before multi-head self-attention.

2.1 Symbol legend

symbol	meaning	typical value
B	batch — scenes per forward pass	1 in this app
N	number of views / images	≈ 1–50
S	tokens per image: $1_{\text{cam}} + 4_{\text{reg}} + P$, with $P = (H/14)(W/14)$	e.g. 5 + 1369 = 1374 at 518×518
C	hidden dim — width of every token vector	1024 (ViT-L/14)
MSA	Multi-head Self-Attention: h parallel heads of $\text{softmax}(QK^\top/\sqrt{d_k})V$, concatenated then linearly projected	$h = 16$ heads, $d_k = C/h = 64$
MLP	Multi-Layer Perceptron (FFN): per-token 2-layer $W_2\,\text{GELU}(W_1 x)$	MLP ratio 4: $C \to 4C \to C$
LN	LayerNorm — normalizes each token to zero mean / unit variance, then learned γ, β	"pre-LN" = LN inside residual branch

L vs h — two integers, different axes

symbol	what it counts	axis	value
L	stacked AA blocks	sequential — depth	≈ 24 in VGGT-1B
h	attention heads inside one MSA	parallel — width within a layer	16 in ViT-L/14

L stacks blocks vertically: block 1 → block 2 → … → block L. h runs horizontally: inside a single MSA, h attentions on $d_k = C/h$-wide slices execute in parallel and are concatenated by W^O. A VGGT forward pass runs roughly $L \times 2 \times h = 24{\cdot}2{\cdot}16 = 768$ head-attention computations (×2 because each AA block has both a frame-wise *and* a global sub-block).

Mental model: L = floors of a building; h = workers on one floor.

2.2 Refresher — LayerNorm (LN)

For a single token vector $\mathbf{x} \in \mathbb{R}^C$ (one of the $B{\cdot}N{\cdot}S$ tokens), LN computes per-token statistics across the channel axis:

$$\mu = \tfrac{1}{C}\sum_{i=1}^{C} x_i, \qquad \sigma^2 = \tfrac{1}{C}\sum_{i=1}^{C}(x_i - \mu)^2$$

then normalizes and applies learned per-channel affine $\gamma, \beta \in \mathbb{R}^C$:

$$\mathrm{LN}(\mathbf{x})_i = \gamma_i \cdot \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_i.$$

LN is applied **independently per token** — it never mixes information across tokens or batch. (Contrast with BatchNorm, which would couple tokens across scenes.) That's why the same LN works unchanged for both AA sub-blocks: the frame-wise reshape $(B \cdot N, S, C)$ and the global reshape $(B, N \cdot S, C)$ both leave the per-token C axis intact.

Pre-LN vs post-LN — why VGGT uses pre-LN

$$\underbrace{\mathbf{x} \leftarrow \mathbf{x} + \mathrm{MSA}(\mathrm{LN}(\mathbf{x}))}_{\textbf{pre-LN — VGGT}} \quad \text{vs.} \quad \underbrace{\mathbf{x} \leftarrow \mathrm{LN}(\mathbf{x} + \mathrm{MSA}(\mathbf{x}))}_{\text{post-LN — original Transformer}}$$

Pre-LN became standard because with deep stacks (24+ blocks) it gives better gradient flow and trains without learning-rate warmup (Xiong et al. 2020). The residual stream can accumulate features across all L layers without LN squashing them every step — important for an aggregator that has to carry geometry information end-to-end.

2.3 Why the residual `+ x`?

Why *add* the sublayer output back to the input rather than *replace* it? Four reasons, ordered by impact. Collectively known as a **residual / skip connection** (ResNet, He et al. 2015; adopted by Transformer, Vaswani et al. 2017).

1 — Gradient flow. Without a residual, stacking L layers makes the back-prop gradient a long product $\prod_\ell f'_\ell(x_\ell)$ that vanishes exponentially with depth. With the residual the local derivative becomes $I + f'_\ell(x_\ell)$ — the I term guarantees a clean gradient highway back to early layers. Essential for a 24-block stack.

2 — Identity-friendly init. At initialization MSA/MLP weights are small/random, so $\mathrm{MSA}(\mathrm{LN}(x))$ is a tiny perturbation. The residual makes $x + \text{small} \approx x$: the stack starts as the identity function and gradually learns useful refinements.

3 — Sublayers learn deltas, not replacements. Rearrange: $\mathrm{MSA}(\mathrm{LN}(x)) = x_{\text{new}} - x_{\text{old}}$. The sublayer's job is to predict "what should I add to refine the current representation?" — much easier than "what is the next representation from scratch?".

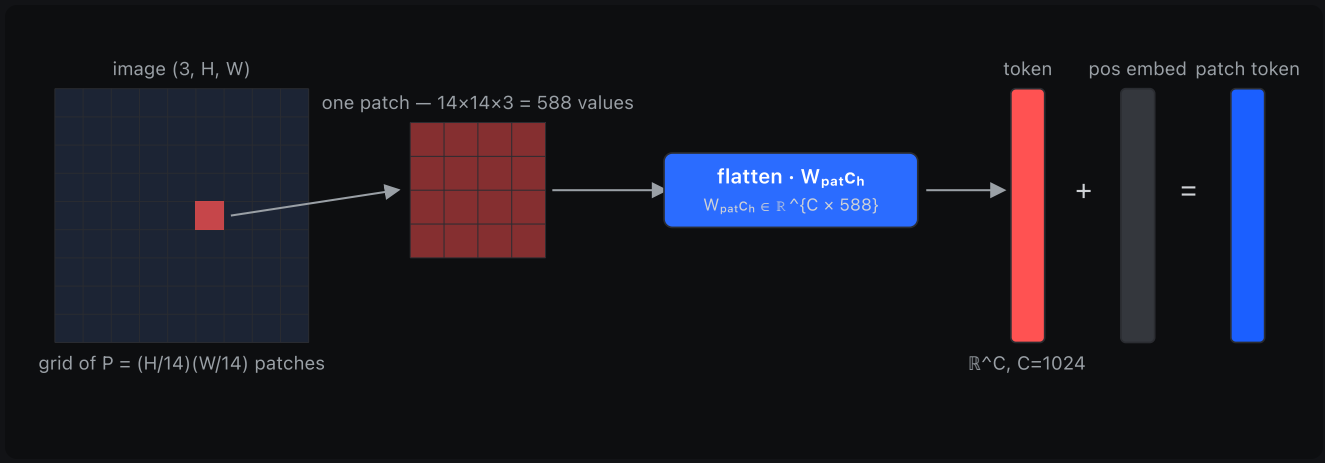
4 — The residual stream — a communication bus. Each sublayer reads from it (via LN), computes something, and writes back by adding. Information accumulates rather than being overwritten. For VGGT this is what lets the aggregator carry geometric structure (position embeds, camera-token signal, depth cues) through all 24 layers.

Net effect: the model behaves like a base signal x plus a sum of learned corrections $\sum_\ell f_\ell(\mathrm{LN}(x_\ell))$.

2.4 One patch → one token (the 1-to-1 mapping)

The image is tiled by the patch-embed conv (kernel = stride = 14) into non-overlapping 14×14×3 tiles. Each tile is compressed by one learned linear projection to a single C-dim token, then a position embedding is added:

$$\underbrace{\text{patch} \in \mathbb{R}^{588}}_{14\times14\times3} \xrightarrow{W_{\text{patch}}\in\mathbb{R}^{C\times588}} \text{token} \in \mathbb{R}^{C} \xrightarrow{+\text{pos embed}} \text{patch token} \in \mathbb{R}^{C}.$$



Stacking all $P = (H/14)(W/14)$ patch tokens gives the patch part of the per-image sequence: a (P, C) matrix. The aggregator then prepends **5 extra tokens that don't come from any patch** — 1 camera + 4 register tokens (learned vectors); together they make the $S = 1 + 4 + P$ per-image sequence.

Two practical consequences: (i) the live attention picker's similarity heatmap is exactly $(H/14)\times(W/14)$ — one cell per patch token, not per pixel; (ii) H, W must be multiples of 14, otherwise patches don't tile the image cleanly.

2.5 Why DINOv2 pretraining matters

W_{patch} (and the early aggregator weights) are **not learned from scratch** — they are initialized from **DINOv2 ViT-L/14** (Oquab et al. 2023), a vision transformer pretrained self-supervised on ~142M images. The "registers" variant gives the 4 register tokens for free.

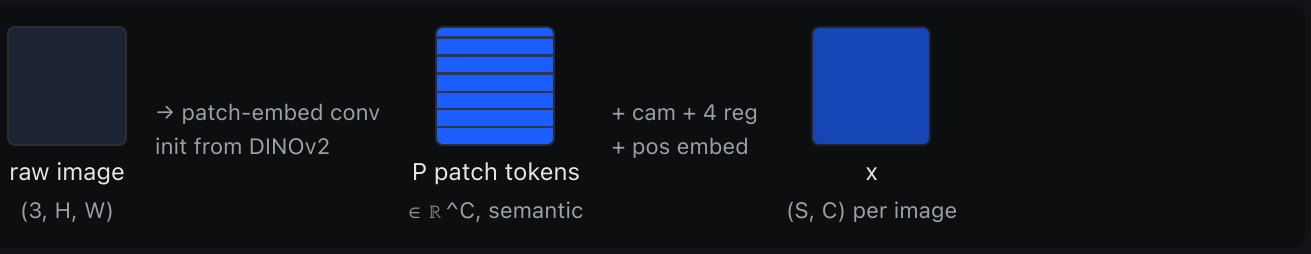
DINOv2's pretraining in one paragraph

A student ViT and a teacher ViT (an exponential moving average of the student) both encode two random crops of the same image. The student is trained so its patch tokens and image-level [CLS] token match the teacher's — without any human labels. The student learns features invariant to crop, scale, and color jitter: patches that depict the same surface end up with similar feature vectors regardless of viewpoint.

What that buys VGGT on day-zero of training

- **Patch tokens already cluster by content.** DINOv2 features out-of-the-box support unsupervised segmentation and retrieval. So two patches across frames that depict the same physical surface already have similar token vectors before any VGGT-specific training has happened — a head-start for the aggregator's global sub-blocks.
- **VGGT learns geometry on top of semantics, not pixels.** The aggregator's job becomes "given semantically-meaningful patch tokens, fuse views and predict cameras + depth" rather than "do everything end-to-end from RGB".
- **Far less 3D-annotated data needed.** DINOv2 absorbed the visual world; VGGT only has to specialize that prior for 3D — which is why a ~1B-param checkpoint trained on ~17 datasets can generalize to in-the-wild scenes.

Visual chain: image → DINOv2 patches → VGGT tokens



2.5b Why 4 register tokens? (the "junk drawer" of the transformer)

The problem registers solve. Train a standard ViT (no registers) and a strange artefact emerges: a small number of patch tokens — usually in *low-information* regions (sky, smooth walls, blurred background) — develop unusually **high norms**. The model has repurposed them as **global scratch space**: that's where it now stores scene-level bookkeeping (statistics, normalizations, "this is an outdoor image", ...) because it has nowhere else to put it.

This hurts dense prediction badly. The patches that *should* represent the sky no longer do — they've been hijacked for bookkeeping. A depth head reading those patches gets garbage exactly where it most needs to rely on a clean prior.

The fix (Darcet et al. 2023, "Vision Transformers Need Registers"): prepend a handful of **dummy tokens** that are not derived from the image — just learned vectors. Now the model has a dedicated scratchpad: it can write its global summaries to the registers instead of hijacking real patches.

After training with registers:

- **Patch tokens stay clean** — low-norm, faithful to their local 14×14 region.
- **Register tokens absorb the bookkeeping** — they're the high-norm outliers now, but they don't represent any image content so no harm done.
- **Attention maps become smooth** — the spotty hot-spots over sky regions disappear.
- **Downstream dense prediction (depth, segmentation, point maps) improves measurably**, especially in textureless regions.

> **Why specifically 4?**
>
> - **0 registers**: artefact appears, dense prediction suffers.
> - **1 register**: helps but not enough — one slot isn't enough scratch space.
> - **4–8 registers**: artefact fully gone, downstream tasks improve.
> - **More**: diminishing returns; just costs compute.
>
> DINOv2 chose 4 as the sweet spot. VGGT inherits this from the with-registers variant.

Why VGGT cares specifically. Two of VGGT's four heads (DPT depth and DPT point) read *patch tokens* directly to predict per-pixel outputs. If patch tokens in textureless regions were corrupted with global bookkeeping, depth there would be unreliable — exactly where you most need a clean prior. Registers keep patch tokens trustworthy.

The heads then **ignore the register tokens entirely** — that's why `ps_idx = 5` exists, to tell the heads "patches start *after* the 5 special tokens". Registers are scratch space; they don't carry information you'd want in the output.

Mental model. Registers are the *junk drawer* of the transformer. Without one, junk piles up on your kitchen counter (real patches). With one, the counter stays clean.

2.6 Per-image token layout (one row of length S)

`cam` = camera token (read by camera head). `r0..r3` = register tokens (absorb high-norm artefacts; $\texttt{ps\_idx} = 5$ tells heads patches start here). p_i = patch tokens, row-major over P.

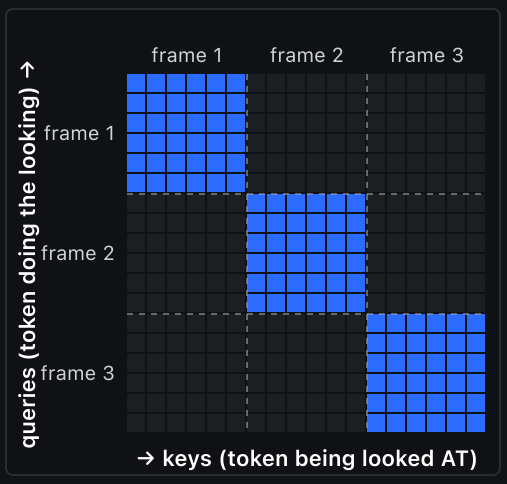
2.7 Step-by-step flow inside one AA block

1. **Input.** $\mathbf{X} \in \mathbb{R}^{B \times N \times S \times C}$ with $C \approx 1024$.
2. **Frame-wise reshape.** $(B, N, S, C) \to (B \cdot N, S, C)$. Each image is now an independent length-S sequence.
3. $\mathrm{MSA}_{\mathrm{frame}}$ **+ residual.** $\mathbf{X} \leftarrow \mathbf{X} + \mathrm{MSA}(\mathrm{LN}(\mathbf{X}))$; 16 heads, mask = block-diagonal. Cost $\mathcal{O}(NS^2C)$.
4. **MLP + residual.** $\mathbf{X} \leftarrow \mathbf{X} + \mathrm{MLP}(\mathrm{LN}(\mathbf{X}))$; MLP ratio 4: $C \to 4C \to C$ with GELU. Reshape back to (B, N, S, C).
5. **Global reshape.** $(B, N, S, C) \to (B, N \cdot S, C)$.
6. $\mathrm{MSA}_{\mathrm{global}}$ **+ residual.** Same MSA with full attention over NS positions. Cost $\mathcal{O}((NS)^2C)$. **This is where view fusion happens.**
7. **MLP + residual; reshape back.** Output again (B, N, S, C), ready for the next AA block.

2.8 Attention-mask comparison (N=3 frames, S=6 tokens/frame demo)

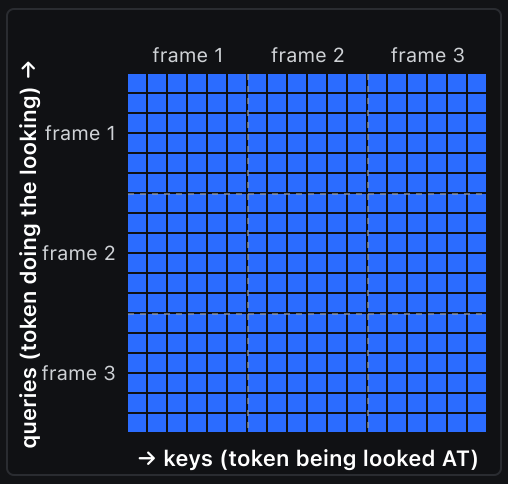
Each grid is the $NS \times NS$ table of "is this query token allowed to look at this key token?" — row = query (token doing the looking), column = key (token being looked at). Each cell is one entry of the score matrix $A = QK^\top$ before softmax. Blue = attention allowed (computed). Grey = blocked / mask 0.

Frame-wise sub-block — only same-frame pairs talk



Blue blocks lie on the diagonal: a query in frame 2 can only attend to keys in frame 2. (Not an explicit mask — the reshape $(B, N, S, C) \to (B \cdot N, S, C)$ puts each frame in its own MSA call.)

Global sub-block — anyone can talk to anyone



The entire $NS \times NS$ grid is blue — a query in frame 2 can pull from keys in frame 1 or 3. This is where view fusion actually happens.

2.9 Refresher — Multi-head Self-Attention (MSA)

Three learned linear projections turn each input token into a query, key, and value. Queries attend to keys (softmax-normalized) to produce weights, which then aggregate values. Doing this in h parallel heads (each on a $d_k = C/h = 64$-dim slice) lets the model attend to several different relations at once:

$$Q = xW^Q, \quad K = xW^K, \quad V = xW^V$$

$$\mathrm{Attn}_i(Q, K, V) = \mathrm{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i$$

$$\mathrm{MSA}(x) = \left[\mathrm{Attn}_1 \,\|\, \ldots \,\|\, \mathrm{Attn}_h\right] W^O$$

Eq 1 — projections

same shape for K = xW^{K} and V = xW^{V} (different weights, identical structure)

Eq 2 — per-head attention

C vs d_k — easy to confuse

C	full hidden dim — model width	1024
d_k	**per-head** dim — slice of C: $d_k = C/h$	$1024/16 = 64$

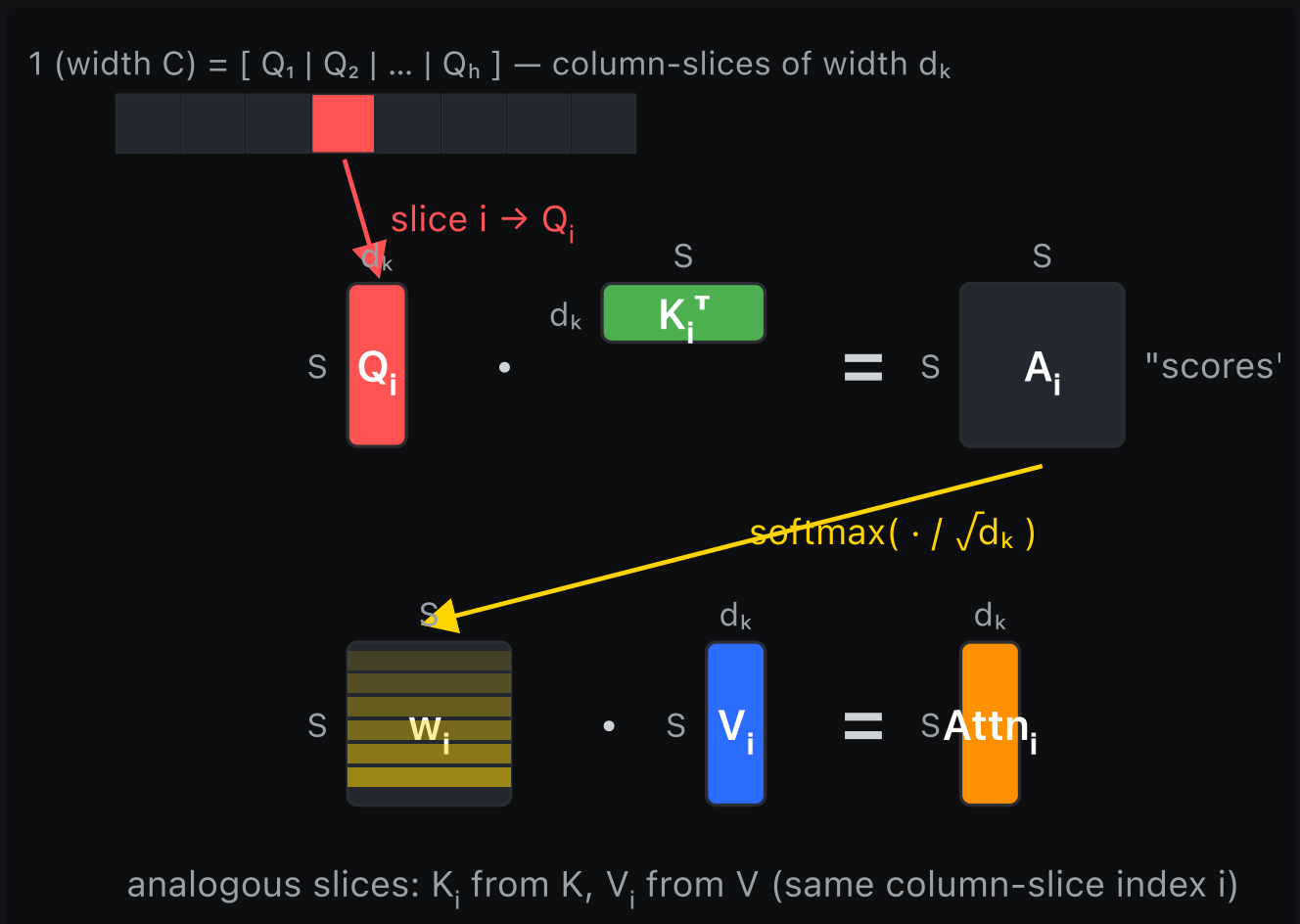
Eq 1 (projection) and Eq 3 (concat + W^O) operate at **full width** C. Eq 2 (per-head attention) operates at the narrower d_k — that's why Q_i, K_i, V_i are drawn as thin bars: each head sees only 64 of the 1024 channels. The score matrix $A_i \in \mathbb{R}^{S\times S}$ has neither C nor d_k in its shape; d_k survives only in the $1/\sqrt{d_k}$ temperature scaling.

Indexing: `i` = head; rows = tokens

Q_i is not "the query for token i" — i is the head index. Each $Q_i \in \mathbb{R}^{S\times d_k}$: rows are all S tokens' queries restricted to head i's 64 channels. So the matmul $Q_i \cdot K_i^\top = A_i$ produces an $S \times S$ matrix where:

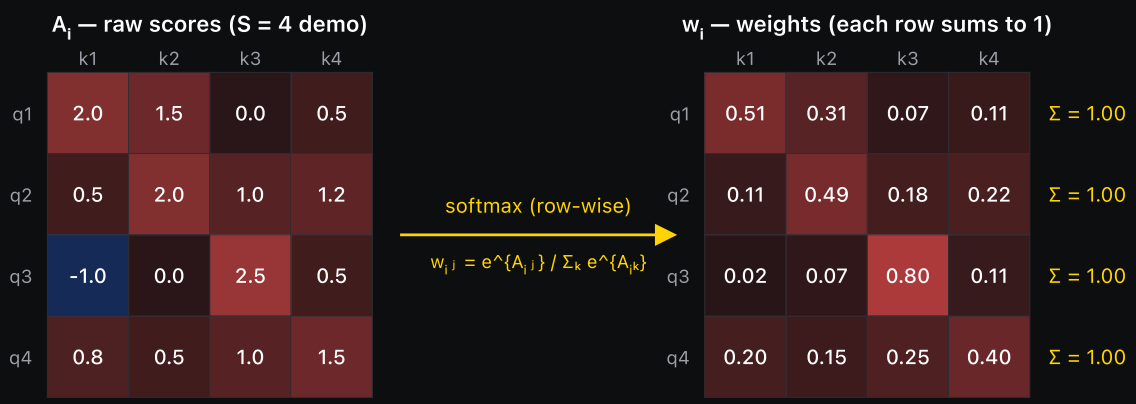
$$A_i[r, c] = \langle q_r,\ k_c \rangle$$

i.e. the dot product between token r's query and token c's key. Row r of A_i is token r's "relevance row" — how much it wants to attend to each of the S tokens.



Worked example — softmax on real numbers

A toy $S = 4$ run. The left grid is the raw score matrix A_i (red = positive, blue = negative dot products). Apply softmax independently to *each row* — exponentiate and divide by the row sum — and you get the right grid w_i, where every row sums to 1.



Row 1: scores $[2.0, 1.5, 0.0, 0.5]$ → softmax $[0.51, 0.31, 0.07, 0.11]$. Row 3's score 2.5 for token 3 dominates → weight 0.80 — softmax exaggerates the winner because the exponential grows fast. That's why softmax is sometimes called a "soft argmax".

Intuitions for each multiplication

$Q_i K_i^\top$ — "how relevant is every token to me?"

Each cell $A_{rc} = \langle q_r, k_c \rangle$ is a dot product (similarity in the learned feature space). Row r is token r's relevance row: for each candidate token, how strongly token r wants to look at it. High score = "I should pay attention here".

softmax — "turn raw votes into a distribution"

Softmax normalizes each row to sum to 1 and emphasizes winners: exp makes big scores big and suppresses small/negative. After softmax, w_{rc} is a probability — fraction of attention budget that token r spends on token c.

$w_i V_i$ — "pull in info from the tokens you chose"

Row r of the output is a weighted average of value vectors:

$$\mathrm{Attn}_i[r] = \sum_{c=1}^{S} w_{rc}\, v_c.$$

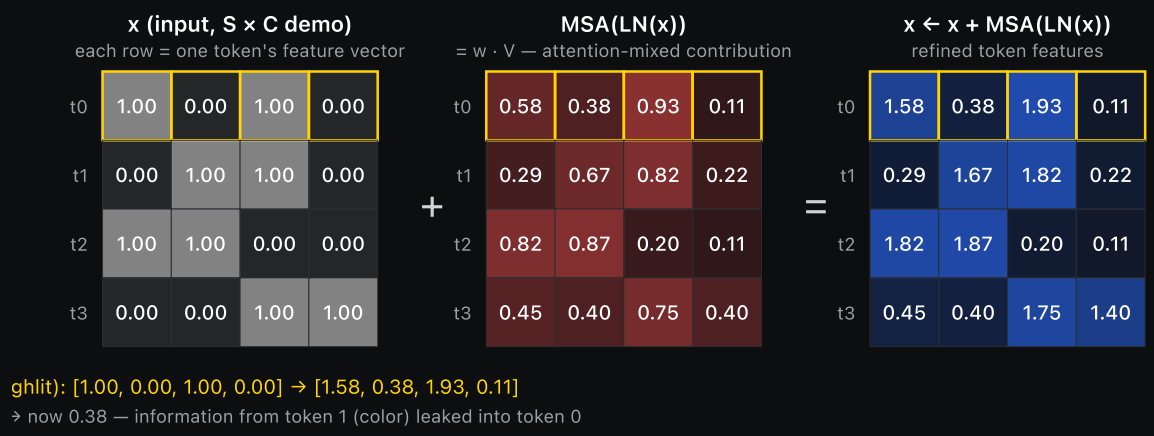
If token 1 attended mostly to token 3 ($w_{13} = 0.8$), its output is roughly $0.8\, v_3 +$ $\text{small contributions from others}$. Each query position effectively reaches out into the sequence and pulls back a tailored mixture of values from whoever it found most relevant.

final $\mathrm{Attn}_i \in \mathbb{R}^{S \times d_k}$ — "every token, now context-aware"

After one attention pass every token contains information from other tokens it deemed relevant. Stack $L \approx 24$ AA blocks and the effect compounds: by deep layers, a patch token has absorbed information from many patches in its own frame (frame-wise) and from patches in other frames (global).

Worked example — how the input tensor changes after MSA

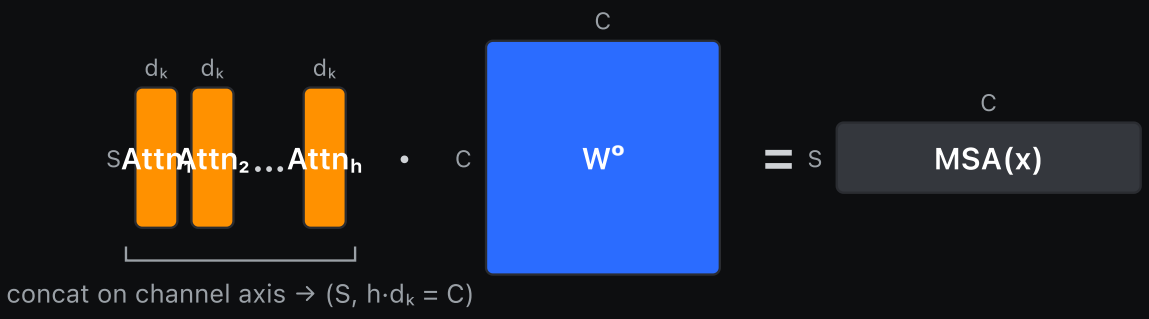
Apply the row-stochastic weights from the softmax example to a tiny input tensor x with $S = 4$ tokens and a 4-dim feature axis. For clarity we set $V_i = x$.



Token 0's original features $[1.0, 0.0, 1.0, 0.0]$ — nothing in dim 1. After the attention sub-block writes its contribution back into the residual, dim 1 of token 0 is 0.38, from token 1 (which had 1.0 in dim 1 and weight $w_{01} = 0.31$). That's context-awareness made concrete: tokens pick up features from other tokens they attended to.

Eq 3 — concat heads + W^O

$$\mathrm{MSA}(x) = \left[\mathrm{Attn}_1 \,\|\, \ldots \,\|\, \mathrm{Attn}_h\right] W^O$$



$W^Q, W^K, W^V, W^O \in \mathbb{R}^{C \times C}$ are the four learned linear layers of MSA. W^O is the **output projection**: after the h heads each produce a (S, d_k) slice, they're concatenated on the channel axis to (S, C) and multiplied by W^O. Without it the heads would never mix — each would only write to its own d_k contiguous channels — so W^O blends them into a single coherent vector before it gets added back into the residual stream.

Why a separate W^V? Why not just $w \cdot x$?

Three reasons VGGT (and every transformer) uses a value projection:

1 — Decoupling routing from payload. Q, K answer "who's relevant to me?"; V answers "what should I deliver?". Tying them to a shared x forces two unrelated roles onto the same weights. A separate W^V lets "what info do I provide?" specialize independently from "what makes me look relevant?".

2 — Per-head specialization. W^V is sliced into h heads, so each head's V_i can extract a different aspect of x: one head might pull color, another edges, another depth cues. Without it every head would deliver the identical payload (raw x).

3 — Without W^V attention is linear in x. Output would be a linear combination of input tokens. Stacking many such layers can only produce more linear combinations. Together with the MLP non-linearity, the learned W^V gives the network the expressivity to learn arbitrary nonlinear features of arbitrary subsets of tokens.

How W^Q, W^K, W^V actually get trained

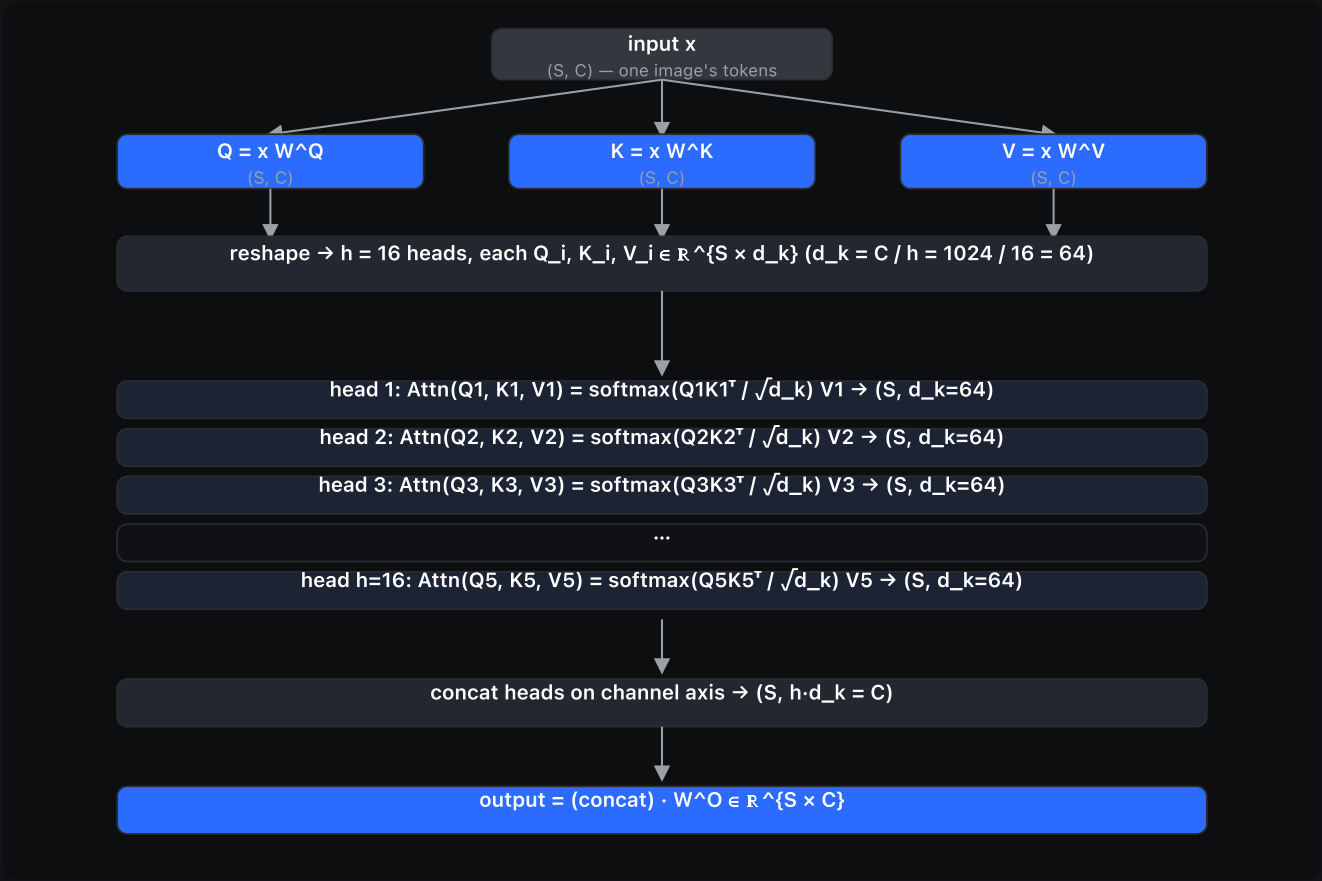
Same recipe as every other weight: **random init → forward pass → loss → backprop → SGD step**. Toy thought experiment: suppose the task says "for each patch, predict the average depth of patches with similar color." Gradient descent will end up discovering that:

- W^Q projects x so q_r encodes "what color am I asking about?"
- W^K projects x so k_c encodes "what color do I have?" — making $q_r \cdot k_c$ large when colors match.
- W^V projects x so v_c encodes token c's depth — the payload to retrieve.

Nobody hand-codes any of this. The optimizer just nudges every entry of every W in whichever direction reduces loss; specialized patterns emerge after millions of steps. Mech-interp work has named some — "induction heads", "previous-token heads". In VGGT, heads end up doing cross-view feature matching for geometry.

In code: PyTorch autograd computes $\partial \mathcal{L}/\partial W^Q$ (and same for W^K, W^V) by the chain rule. AdamW then updates $W \leftarrow W - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$.

2.10 Putting it together — full block flow



2.11 Refresher — MLP (FFN inside each block)

Applied **independently to every token** (no token mixing — that job belongs to MSA). It's a 2-layer feed-forward net with 4× channel expansion and a GELU non-linearity:

$$\mathrm{MLP}(x) = W_2\,\mathrm{GELU}(W_1 x), \qquad W_1 \in \mathbb{R}^{4C\times C},\ W_2 \in \mathbb{R}^{C\times 4C}.$$

Roughly two-thirds of a transformer block's parameters live in this expansion ($2 \cdot C \cdot 4C = 8C^2$ for MLP vs $4C^2$ for Q, K, V, W^O combined).

GELU vs ReLU — and why every modern transformer picks GELU

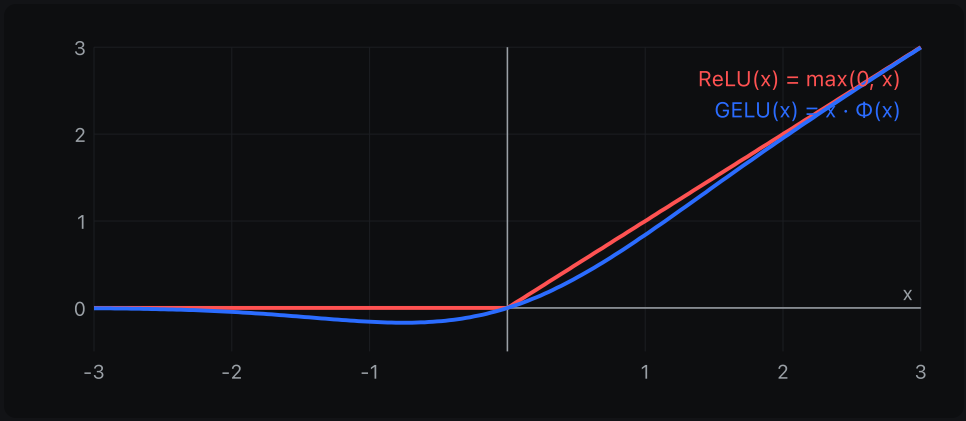
GELU (Gaussian Error Linear Unit), Hendrycks & Gimpel 2016:

$$\mathrm{GELU}(x) = x \cdot \Phi(x) = x \cdot \tfrac{1}{2}\left[1 + \mathrm{erf}\left(\tfrac{x}{\sqrt{2}}\right)\right]$$

where $\Phi(x)$ is the cumulative density of the standard normal. In practice almost everyone uses the cheap tanh-based approximation:

$$\mathrm{GELU}(x) \;\approx\; 0.5\,x\left[1+\tanh\!\left(\sqrt{\tfrac{2}{\pi}}\left(x+0.044715\,x^3\right)\right)\right] \;\approx\; x\cdot\sigma(1.702\,x)$$

Visually:



Why GELU beats ReLU (for transformers specifically):

1 — Smooth and differentiable everywhere. ReLU has a hard kink at $x = 0$ where the derivative jumps from 0 to 1. That non-smoothness makes optimization landscapes rougher and second-order info noisier. GELU is C^∞-smooth — better behaved for the optimizers (AdamW, LAMB) and for any layer-norm / initialization analysis that assumes smoothness.

2 — No "dying ReLU". When a ReLU unit's input is consistently negative, $\mathrm{ReLU}'(x) = 0$ → no gradient → the neuron stops learning forever ("dead neuron"). GELU has small *non-zero* output and gradient for negative inputs (especially around $x \approx -1$), so units recover.

3 — Stochastic gating without stochasticity. The probabilistic reading: $\mathrm{GELU}(x) = \mathbb{E}_{m\sim\mathrm{Bernoulli}(\Phi(x))}[m\cdot x]$. GELU is the *expected* output of "randomly keep x with probability $\Phi(x)$ (its percentile rank in a standard normal), else drop it to zero." So it's a deterministic activation that **bakes in a soft data-dependent dropout**. Empirically this acts as a regularizer and helps generalization.

4 — Negative dip ≈ subtle inhibition. Notice the small *negative* GELU values for $x \in [-1.5, 0]$ (visible in the plot). The activation can push a token's feature slightly negative — a form of "soft inhibition" that ReLU literally cannot represent. Empirically this small expressivity bump matters at scale.

Adopted by BERT, GPT-2/3/4, ViT, DINOv2 — basically every modern transformer — after Hendrycks & Gimpel's NLP benchmark and follow-ups showed consistent (small but real) gains over ReLU and ELU. The cost over ReLU is one extra `tanh` per activation, which CUDA buries.

What is the MLP actually for? (intuition)

If MSA is the **communication** step ("look around, gather info"), the MLP is the **processing / thinking** step ("now do something with what you gathered").

1 — Per-token nonlinear refinement. Attention produced a token whose value is a linear weighted sum of values from elsewhere. MLP applies a non-linear transformation per token — it can detect "is the mixed feature now a corner?", "is it a depth discontinuity?" — things that need non-linear thresholds on combinations of features.

2 — Feature detectors / knowledge storage. The $4C = 4096$ hidden units are independent feature detectors. Mech-interp research finds that MLPs are where transformers store features and facts; attention is where they route them. In VGGT terms: attention says "patch r and patch c are the same surface"; the MLP then computes new geometry-aware features from that paired signal.

3 — Required for depth to do anything. A stack of MSA-only blocks would collapse to a single linear-plus-softmax function — no matter how deep, you wouldn't gain expressivity. The per-token MLP non-linearity is what makes the depth $L = 24$ useful.

Block-level recipe: gather (MSA) → think (MLP) → pass to next block. After 24 of these, every patch has had 24 rounds of "look around, then think" — gathering context from across frames and refining it through 24 layers of non-linear feature construction.

MLP parallelism — what "token-wise" actually means

The MLP is the **one place** in a transformer block where tokens are processed **fully in parallel and independently**. Given input $X \in \mathbb{R}^{S \times C}$, for each token $\mathbf{x}_s$ independently:

$$\mathbf{y}_s = W_2 \,\mathrm{GELU}(W_1 \mathbf{x}_s + b_1) + b_2, \quad s = 1, \ldots, S.$$

Note the absence of any s index on the weights — the *same* W_1, W_2 are applied to every token. Token 1's output depends only on token 1's input. **There is no cross-token interaction at all.**

In code it's not a Python loop — it's literally one big matmul:

```
# X has shape (S, C)  — or after a frame-wise reshape, (B*N, S, C)
H = (X @ W1.T).gelu()      # (S, 4C)  one matmul, all S tokens at once
Y = H @ W2.T               # (S, C)   one matmul, all S tokens at once
```

CUDA cores process every row (every token) in parallel — that's where the throughput comes from. After the frame-wise reshape to $(B \cdot N, S, C)$, all $B \cdot N \cdot S$ tokens flow through the MLP in one tensor op.

Contrast with attention

	per-token (no info flow)?	parallel?
MSA	No — each token's output depends on *every* other token through $Q \cdot K^\top$	Yes, but with explicit $S \times S$ interaction
MLP	**Yes** — each token transformed in isolation	Yes — embarrassingly so; just S independent function applications batched into one matmul

Why this clean split matters. Attention = communication (cross-token information flow, parameterized by W^Q, W^K, W^V, W^O). **MLP = computation** (per-token non-linear processing, parameterized by W_1, W_2). If the MLP also mixed tokens, you'd have two cross-token operations per block — redundant and harder to interpret. If attention also did non-linear feature transformation, you'd lose the clean separation. The split is half of why transformers work so well.

One subtle consequence: the MLP behaves **identically** in the frame-wise and global sub-blocks, because the reshape from $(B \cdot N, S, C)$ to $(B, N \cdot S, C)$ doesn't affect a per-token operation. The "frame-wise vs global" distinction is purely about *attention* — the only layer that does cross-token computation.

3. Geometry math (with refreshers)

3.1 Pinhole projection

A world point $\mathbf{X}_w$ maps to a pixel via extrinsics then intrinsics:

$$\mathbf{X}_c = R\,\mathbf{X}_w + \mathbf{t}, \qquad \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim K\,\mathbf{X}_c, \quad K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

VGGT puts the principal point at the image center $(c_x, c_y) = (W/2, H/2)$ and derives focal from FOV:

$$f_x = \frac{W/2}{\tan(\mathrm{FOV}_x/2)}.$$

3.2 Rotation: quaternion → R

The pose encoding stores rotation as a unit quaternion $\mathbf{q} = (w, x, y, z)$. It expands to:

$$R = \begin{bmatrix} 1-2(y^2+z^2) & 2(xy-wz) & 2(xz+wy) \\ 2(xy+wz) & 1-2(x^2+z^2) & 2(yz-wx) \\ 2(xz-wy) & 2(yz+wx) & 1-2(x^2+y^2) \end{bmatrix}.$$

3.3 Depth unprojection (what builds the cloud)

For pixel (u, v) with predicted depth d, the world point is the inverse pipeline:

$$\mathbf{X}_w = R^{\top}\big(d\,K^{-1}[u, v, 1]^{\top} - \mathbf{t}\big).$$

This is exactly what `unproject_depth_map_to_point_map` computes. In the live app the geometry-math sliders let you change the FOV and depth interactively and watch the frustum + principal-ray update on the 3D view — that interactive part is omitted from this PDF.

4. Training (conceptual — code & data mixture not publicly released)

4.1 Multi-task loss

VGGT is trained end-to-end with a weighted sum over the heads:

$$\mathcal{L} = \mathcal{L}_{\text{cam}} + \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{point}} + \lambda\,\mathcal{L}_{\text{track}}.$$

Aleatoric (Laplace) confidence weighting — a refresher

The dense heads predict an uncertainty σ; the loss down-weights uncertain pixels and pays a small price for being uncertain (learned, per-pixel robustness):

$$\mathcal{L}_{\text{depth}} = |\hat{y} - y| \cdot e^{-\sigma} + \alpha\,\sigma.$$

Camera loss: Huber on the pose-encoding components $(\mathbf{t}, \mathbf{q}, \text{FOV})$. Track loss: L1 on tracks + BCE on visibility (CoTracker-style).

4.2 Recipe (from the paper)

- Backbone initialized from **DINOv2**.
- Variable views per sample (≈1–24) and variable resolution — teaches view-count generalization.
- AdamW with cosine learning-rate schedule; large-scale (paper reports ~64×A100 GPUs).
- Ground truth = real + synthetic datasets with metric depth and known camera poses.

4.3 Indicative dataset mixture

The paper's exact mix is not publicly released. An indicative composition from public information:

dataset	indicative share
Co3Dv2	~18%
BlendedMVS	~14%
MegaDepth	~12%
ScanNet++	~11%
ARKitScenes	~10%
Hypersim (synthetic)	~9%
Habitat / HM3D (synthetic)	~9%
TartanAir (synthetic)	~8%
WildRGB-D / others	~9%

⚠ Training code and the exact data mixture are **not publicly released**. The shares above are an indicative composition, not an official breakdown. Nothing here is reproduced — it's a recipe summary from the paper.